

Critical Section Solution using "Lock"

Lock $\neq 0$

1. While (Lock == 1);

ENTRY
CODE

2. LOCK = 1

3. Critical Section

4. LOCK = 0

EXIT
CODE

Case 1

P₁ P₂ X

1234 | P₂ ✓

Lock = 0 → Vacant

1 → full

Lock = 0 + 0, 1

So this way we generally use the lock funda but if you are asked is there any guarantee?

Critical Section Solution using "Lock"

do {
 acquire lock
 CS
 release lock
}

Execute in User Mode
Multiprocess Solution
Mutual Exclusion
entee

Case 1
 $P_1, P_2 \times$
1234 | $P_2 \checkmark$


Lock $\neq 0$

1. While (LOCK == 1);

2. LOCK = 1

ENTRY CODE

3. Critical Section

4. LOCK = 0

EXIT CODE

Lock = 0 $\rightarrow$ Vacant
1 $\rightarrow$ full
Lock = $\emptyset \times \emptyset$

So this way we generally use the lock funda but if you are asked is there any guarantee?

Critical Section Solution using "Lock"

```
do {  
    acquire lock  
    CS  
    release lock  
}
```

- * Execute in User Mode
- * Multiprocess Solution
- * No Mutual Exclusion Guarantee

Case 1
 $P_1, P_2 \times$
1234 | $P_2 \checkmark$


Lock $\neq 0$

1. While (LOCK == 1);

2. LOCK = 1

ENTRY CODE

3. Critical Section

4. LOCK = 0

EXIT CODE

Lock = 0 $\rightarrow$ Vacant
1 $\rightarrow$ full
Lock = $\emptyset$ & $\emptyset$

Look here, is there any guarantee that mutual exclusion will always be there?

Critical Section Solution using "Lock"

```
do {  
    acquire lock  
    CS  
    release lock  
}
```

- * Execute in User Mode
- * Multiprocess Solution
- * No Mutual Exclusion Guarantee

Case 1
 $P_1, P_2 \times$
1234 | $P_2 \checkmark$
 $\boxed{P_1}$
 P_2

Lock $\neq 0$

1. While (LOCK == 1);

2. LOCK = 1

ENTRY CODE

3. Critical Section

4. LOCK = 0

EXIT CODE

Lock = 0 $\rightarrow$ Vacant
1 $\rightarrow$ full
Lock = $\emptyset$ & $\emptyset$

Look here, is there any guarantee that mutual exclusion will always be there?

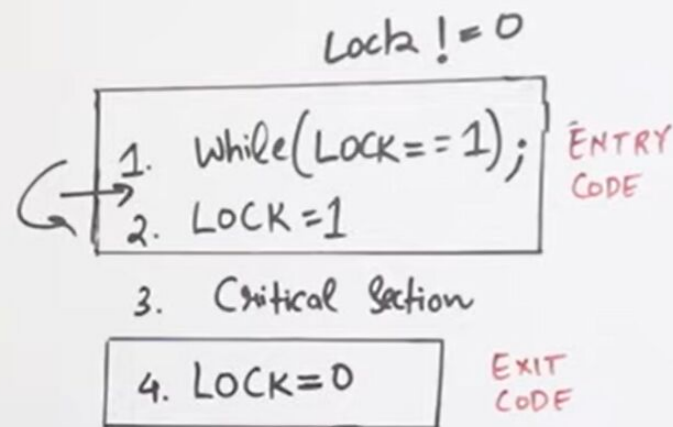
Critical Section Solution using "Lock"

do {
 acquire lock
 CS
 release lock
}

Execute in User Mode

Multiprocess Solution

- * No Mutual Exclusion Guarantee



Case 2

P₁ P₂

L = 0

1 2 3 | 1 2 3
CS

P₂ P₁
CS

So this actually is a problem in lock variable because in between these two if pre-emption happens

Critical Section Solution using "Lock"

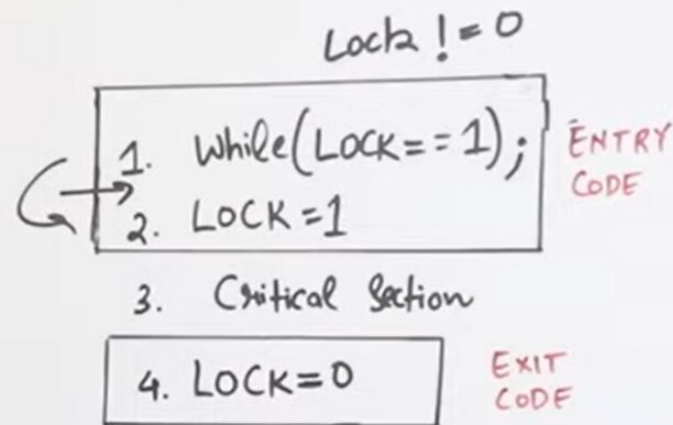
```
do {
    acquire lock
    CS
    release lock
}
```

- * Execute in User Mode
- * Multiprocess Solution
- * Mutual Exclusion
- free

Case 2

$P_1 \quad P_2$
 $\underline{1*23} \mid 123$
 CS

$L = 0 \times 1$
 $\boxed{P_2 P_1}$
 CS



kernel and operating system has nothing to do. We use it for multiprocess and

Critical Section Solution using "Lock"

acquire lock
CS
release lock

ate in User Mode
Multiprocess Solution

* No Mutual Exclusion
Guarantee

Case 2

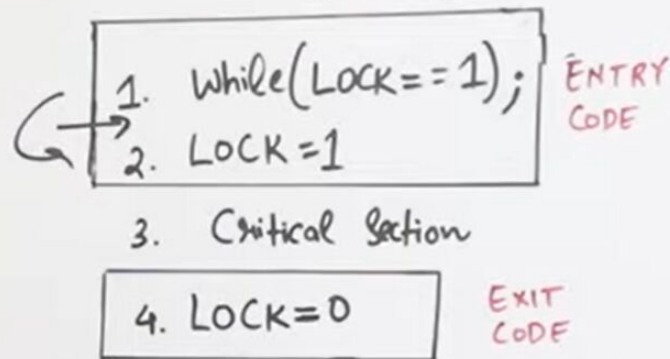
P₁ P₂

$\frac{1*23}{\sqrt{\quad}} \mid \frac{123}{CS}$

$L = \cancel{0} \times 1$

$\boxed{P_2 P_1}$
CS

Lock $\neq 0$



mutual exclusion is not guaranteed. So this is all about, and this is more than sufficient

Critical Section Solution using "Test-and-Set" Instruction

While (test_and_set(&lock));

CS P₁ P₂ X P₁ P₂

lock = false;

T=1
f=0
L=false

1. While(LOCK==1);
2. LOCK=1

ENTRY CODE

3. Critical Section P₂ ← P₁ ←

4. LOCK=0

EXIT CODE

boolean test_and_set (boolean *target)

```
{
    boolean r1 = *target;
    *target = TRUE;
    return r1;
}
```

ME ✓
Progr ✓

ME X

P₁ | P₂ ✓
1 * 2 | 1

lock false
1000
True True

target 1000

r1
True

L = 0 + 1
Lock = 0